

Modelos de Computación

Tema 1: Introducción a la Computación

Salvador Gutiérrez
salvaguti@ugr.es
Curso 2026/2027

Contenido

Breve introducción histórica a la computación

Definiciones básicas y operaciones

Gramáticas

Jerarquía de Chomsky

Contenido

Breve introducción histórica a la computación

Definiciones básicas y operaciones

Gramáticas

Jerarquía de Chomsky

La computación

Vamos a tratar de responder varias preguntas:

- ¿Qué puede ser resuelto de forma automática?
- ¿Qué puede ser resuelto de forma eficiente?
- ¿Qué estructuras son comunes en la computación con palabras y símbolos y cómo se pueden procesar en un ordernador?

La computación

Vamos a tratar de responder varias preguntas:

- ¿Qué puede ser resuelto de forma automática?
- ¿Qué puede ser resuelto de forma eficiente?
- ¿Qué estructuras son comunes en la computación con palabras y símbolos y cómo se pueden procesar en un ordernador?

Consecuencia: *Estudio de Modelos de Computación*

Perspectiva histórica: el problema del cálculo mecánico

- Antes de los ordenadores, la pregunta era puramente matemática: *¿qué problemas pueden resolverse siguiendo un método mecánico, paso a paso, sin necesidad de intuición o ingenio?*
- David Hilbert la formalizó en 1928 junto a Wilhelm Ackermann como el *Entscheidungsproblem* ("problema de decisión"): ¿existe un procedimiento que determine, para cualquier enunciado lógico, si es demostrable?
- Responder exigía primero definir con precisión qué es un *procedimiento mecánico* — de ahí nace la noción de algoritmo, decidibilidad y cálculo formal que vertebra esta asignatura.

Los precursores: lógica y fundamentos (s. XIX – 1931)

- *George Boole* (1854): *Las leyes del pensamiento*. Reduce la lógica a álgebra: primer paso para tratar el razonamiento de forma puramente simbólica.
- *Frege, Russell y Whitehead* (1879–1913): intento de fundamentar toda la matemática en la lógica (*Principia Mathematica*).
- *Hilbert* (años 1920): su "programa" quiere demostrar que las matemáticas son completas, consistentes y decidibles.
- *Kurt Gödel* (1931): sus teoremas de incompletitud muestran que ningún sistema formal suficientemente potente puede ser a la vez completo y demostrar su propia consistencia. El programa de Hilbert es, en parte, imposible.
- Gödel no resuelve el *Entscheidungsproblem*, pero deja claro que la respuesta requiere antes una definición rigurosa de "método mecánico".

1936: nace la noción de algoritmo

- *Alonzo Church* (1936): el *cálculo* λ como modelo de "función efectivamente calculable". Demuestra que el *Entscheidungsproblem* es indecidible.
- *Alan Turing* (1936): define la *máquina de Turing* y llega, por un camino distinto, a la misma conclusión negativa.
- *Emil Post* (1936): formula de manera independiente un modelo de cálculo equivalente ("Formulation 1"), a menudo eclipsado por Church y Turing.
- En 1937, el propio Turing demuestra que su máquina y el cálculo λ de Church son *equivalentes*: nace la *tesis de Church-Turing* — todo lo "efectivamente calculable" es calculable por una máquina de Turing.
- Por primera vez, *calcular* tiene una definición matemática precisa.

De la teoría a la máquina (años 40)

- *McCulloch y Pitts* (1943): modelan neuronas como interruptores lógicos; su "red neuronal" es, en esencia, un autómata finito *antes* de que existiera esa palabra.
- *John von Neumann* (1945): en su informe sobre el EDVAC describe la arquitectura de programa almacenado que todavía hoy domina — inspirada directamente en la idea de *máquina universal* de Turing.
- El ENIAC (1945) y sus sucesores convierten por primera vez estas ideas abstractas en máquinas físicas reales.

Lenguajes formales y autómatas (años 50)

- *Kleene* (1956): demuestra que los lenguajes reconocidos por las "redes nerviosas" de McCulloch-Pitts son exactamente los descritos por *expresiones regulares* — el teorema que da nombre al Tema 2.
- *Noam Chomsky* (1956, 1959): propone una jerarquía de gramáticas (Tipo 0–3) para describir lenguajes naturales, que resulta ser exactamente el marco que necesita la informática para describir lenguajes de programación.
- *Rabin y Scott* (1959): introducen el *no determinismo* en los autómatas finitos y demuestran que no añade poder de reconocimiento — resultado central del Tema 2.
- *Backus* (1959): la notación BNF, creada para especificar la sintaxis de ALGOL, es gramáticas independientes del contexto aplicadas a un lenguaje de programación real.
- Mismos años: FORTRAN (1957, cálculo científico), LISP (1958, McCarthy, inteligencia artificial), COBOL (1959, gestión) — los primeros compiladores ya se apoyan en autómatas y gramáticas.

La era de la complejidad (años 60–70)

- *Edmonds* (1965): propone identificar "eficiente" con "tiempo polinómico" — germen de la clase P.
- *Hartmanis y Stearns* (1965): fundan formalmente la teoría de la complejidad computacional, midiendo el coste en tiempo y espacio de un algoritmo.
- *Cook* (1971) y Levin (1973, en la URSS, de forma independiente): teorema de Cook-Levin, primer problema NP-completo (SAT).
- *Karp* (1972): otros 21 problemas NP-completos — P versus NP se convierte en la pregunta abierta más importante de la disciplina.
- *Scott y Strachey* (años 70): semántica denotacional, un significado matemático riguroso para los programas.
- En paralelo: maduran las estructuras de datos que hoy se dan por sentadas — árboles, grafos, pilas, colas.

Nuevas fronteras (años 80–90)

- **Complejidad paralela y distribuida:** clases como NC (Pippenger, años 70–80) capturan qué problemas se paralelizan bien; *Lamport* sienta las bases teóricas de los sistemas distribuidos (relojes lógicos, 1978; consenso bizantino, 1982).
- **Criptografía:** aunque la clave pública nace antes (*Diffie-Hellman*, 1976; *RSA*, 1977), es en los 80–90 cuando se desarrollan sus fundamentos teóricos — pruebas de conocimiento cero (*Goldwasser-Micali-Rackoff*, 1985) — y su despliegue práctico (SSL, mediados de los 90).
- Conexión con el resto de la asignatura: todo esto se apoya en la jerarquía de Chomsky, la decidibilidad y la teoría de la complejidad que se acaban de introducir.

El problema de la parada

¿Existe un programa que lea un programa y nos diga si ese programa termina o cicla indefinidamente?

El problema de la parada

¿Existe un programa que lea un programa y nos diga si ese programa termina o cicla indefinidamente?

No existe, ya que si existiera `Stops(P)` podríamos construir el algoritmo `Turing(P)` con entrada `P`.

```
L           If Stops(P)
                GOTO L
```

El problema de la parada

¿Existe un programa que lea un programa y nos diga si ese programa termina o cicla indefinidamente?

No existe, ya que si existiera `Stops(P)` podríamos construir el algoritmo `Turing(P)` con entrada `P`.

```
L           If Stops(P)
                GOTO L
```

¿Cual es el resultado de `Turing(Turing)`?

Contenido

Breve introducción histórica a la computación

Definiciones básicas y operaciones

Gramáticas

Jerarquía de Chomsky

Alfabetos

Un **alfabeto** es un conjunto finito A. Sus elementos se llamarán símbolos o letras.

Notación - *Alfabetos:* $A, B, C, \ldots$

Símbolos: $a, b, c, \ldots$ o números.

Ejemplos

- $A = \{0, 1\}$
- $B = \{< 0, 0 >, < 0, 1 >, < 1, 0 >, < 1, 1 >\}$

Palabras

Una **palabra** (o **cadena**) sobre el alfabeto A es una sucesión finita de elementos de A.

$$u = a_1 \dots a_n$$

donde $a_i \in A$, $\forall i = 1, \dots, n$.

Ejemplo

Si $A = \{0, 1\}$ entonces 0111 es una palabra sobre este alfabeto.

Palabras

El conjunto de todas las palabras sobre un alfabeto A se nota como A^*.

Notación - *Palabras:* $u, v, x, y, z, \ldots$.

Palabras

El conjunto de todas las palabras sobre un alfabeto A se nota como A^*.

Notación - *Palabras:* $u, v, x, y, z, \ldots$.

Si $u \in A^*$, entonces la *longitud* de la palabra u es el número de símbolos de A que contiene.

Notación: $|u|$

Si $u = a_1 \ldots a_n$, entonces $|u| = n$.

Palabras

El conjunto de todas las palabras sobre un alfabeto A se nota como A^*.

Notación - *Palabras:* $u, v, x, y, z, \ldots$.

Si $u \in A^*$, entonces la *longitud* de la palabra u es el número de símbolos de A que contiene.

Notación: $|u|$

Si $u = a_1 \ldots a_n$, entonces $|u| = n$.

La *palabra vacía* es la palabra de longitud cero.

Notación: ϵ

Palabras

El conjunto de todas las palabras sobre un alfabeto A se nota como A^*.

Notación - *Palabras:* $u, v, x, y, z, \ldots$.

Si $u \in A^*$, entonces la *longitud* de la palabra u es el número de símbolos de A que contiene.

Notación: $|u|$

Si $u = a_1 \ldots a_n$, entonces $|u| = n$.

La *palabra vacía* es la palabra de longitud cero.

Notación: ϵ

Notación: El conjunto de cadenas sobre un alfabeto A excluyendo la cadena vacía se nota como A^+.

Operaciones: concatenación

Si $u, v \in A^*$, $u = a_1 \dots a_n$, $v = b_1 \dots b_m$, se llama **concatenación** de u y v a la cadena uv dada por $a_1 \dots a_n b_1 \dots b_m$.

Operaciones: concatenación

Si $u, v \in A^*$, $u = a_1 \dots a_n$, $v = b_1 \dots b_m$, se llama **concatenación** de u y v a la cadena uv dada por $a_1 \dots a_n b_1 \dots b_m$.

Ejemplo

Si $u = 011, v = 1010$, entonces $uv = 0111010$

Operaciones: concatenación

Si $u, v \in A^*$, $u = a_1 \dots a_n$, $v = b_1 \dots b_m$, se llama **concatenación** de u y v a la cadena uv dada por $a_1 \dots a_n b_1 \dots b_m$.

Ejemplo

Si $u = 011, v = 1010$, entonces $uv = 0111010$

Propiedades

1. $|uv| = |u| + |v|, \quad \forall u, v \in A^*$
2. *Asociativa.* $u(vw) = (uv)w, \quad \forall u, v, w \in A^*$
3. *Elemento Neutro.* $u\epsilon = \epsilon u = u, \quad \forall u \in A^*$

Prefijos y sufijos

Prefijo

Si $u \in A^*$ entonces $v \in A^*$ es un prefijo de u si $\exists z \in A^*$ tal que $vz = u$

Un prefijo v de u se dice propio si $v \neq \epsilon$ y $v \neq u$.

Sufijo

Si $u \in A^*$ entonces $v \in A^*$ es un sufijo de u si $\exists z \in A^*$ tal que $zv = u$

Un sufijo v de u se dice propio si $v \neq \epsilon$ y $v \neq u$.

Iteración y cadena inversa

Iteración n-ésima de una cadena (u^n) como la concatenación con ella misma n veces.

Si $u \in A^*$ entonces

- $u^0 = \epsilon$
- $u^{i+1} = u^i u, \quad \forall i \geq 0$

Ejemplo

Si $u = 010$, entonces $u^3 = 010010010$.

Iteración y cadena inversa

Iteración n-ésima de una cadena (u^n) como la concatenación con ella misma n veces.

Si $u \in A^*$ entonces

- $u^0 = \epsilon$
- $u^{i+1} = u^i u, \quad \forall i \geq 0$

Ejemplo

Si $u = 010$, entonces $u^3 = 010010010$.

Si $u = a_1 \dots a_n \in A^*$, entonces la *cadena inversa* de u

es la cadena $u^{-1} = a_n \dots a_1 \in A^*$.

Ejemplo

Si $u = 011$, entonces $u^{-1} = 110$.

Lenguajes

Un **lenguaje** sobre el alfabeto A es un subconjunto del conjunto de las cadenas sobre A: $L \subseteq A^*$.

Lenguajes

Un **lenguaje** sobre el alfabeto A es un subconjunto del conjunto de las cadenas sobre A: $L \subseteq A^*$.

Notación - *Lenguajes:* $L, M, N, \ldots$.

Ejemplos:

- $L_1 = \{a, b, \epsilon\}$ *Tres palabras*
- $L_2 = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$
- $L_3 = \{uu^{-1} \mid u \in A^*\}$
- $L_4 = \{a^{n^2} \mid n = 1, 2, 3, \ldots\}$

Lenguajes

Un **lenguaje** sobre el alfabeto A es un subconjunto del conjunto de las cadenas sobre A: $L \subseteq A^*$.

Notación - *Lenguajes:* $L, M, N, \ldots$.

Ejemplos:

- $L_1 = \{a, b, \epsilon\}$ *Tres palabras*
- $L_2 = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$ Una sucesión de a seguida de una de b de la misma longitud
- $L_3 = \{uu^{-1} \mid u \in A^*\}$
- $L_4 = \{a^{n^2} \mid n = 1, 2, 3, \ldots\}$

Lenguajes

Un **lenguaje** sobre el alfabeto A es un subconjunto del conjunto de las cadenas sobre A: $L \subseteq A^*$.

Notación - *Lenguajes:* $L, M, N, \ldots$.

Ejemplos:

- $L_1 = \{a, b, \epsilon\}$ *Tres palabras*
- $L_2 = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$ Una sucesión de a seguida de una de b de la misma longitud
- $L_3 = \{uu^{-1} \mid u \in A^*\}$ *Palíndromos de longitud par*
- $L_4 = \{a^{n^2} \mid n = 1, 2, 3, \ldots\}$

Lenguajes

Un **lenguaje** sobre el alfabeto A es un subconjunto del conjunto de las cadenas sobre A: $L \subseteq A^*$.

Notación - *Lenguajes:* $L, M, N, \ldots$.

Ejemplos:

- $L_1 = \{a, b, \epsilon\}$ *Tres palabras*
- $L_2 = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$ Una sucesión de a seguida de una de b de la misma longitud
- $L_3 = \{uu^{-1} \mid u \in A^*\}$ *Palíndromos de longitud par*
- $L_4 = \{a^{n^2} \mid n = 1, 2, 3, \ldots\}$ Sucesiones de a de longitud un cuadrado perfecto

Operaciones

Aparte de las operaciones de unión e intersección de lenguajes, dada su condición de conjuntos existe la operación de concatenación.

Si L_1, L_2 son dos lenguajes sobre el alfabeto A, la **concatenación** de estos dos lenguajes se define como,

$$L_1 L_2 = \{u_1 u_2 \mid u_1 \in L_1, u_2 \in L_2\}$$

Propiedades

- $L\emptyset = \emptyset L = \emptyset$
- *Elemento Neutro.* $\{\epsilon\}L = L\{\epsilon\} = L$
- *Asociativa.* $L_1(L_2 L_3) = (L_1 L_2)L_3$

Ejemplo

Si $L_1 = \{0^i 1^i \ : \ i \geq 0\}, \quad L_2 = \{1^i 0^i \ : \ i \geq 0\}$ entonces,

$L_1 L_2 =$

Ejemplo

Si $L_1 = \{0^i 1^i \ : \ i \geq 0\}, \quad L_2 = \{1^i 0^i \ : \ i \geq 0\}$ entonces,

$L_1 L_2 = \{0^i 1^i 1^j 0^j \ : \ i, j \geq 0\}$

Operaciones

La **iteración** de lenguajes se define de forma recursiva:

$$L^0 = \{\epsilon\}, \qquad L^{i+1} = L^i L$$

Si L es un lenguaje sobre el alfabeto A, la *clausura de Kleene* de L es

$$L^* = \bigcup_{i \geq 0} L^i$$

$$L^+ = \bigcup_{i \geq 1} L^i$$

Operaciones

Propiedades:

- $L^+ = L^*$ si $\epsilon \in L$
- $L^+ = L^* - \{\epsilon\}$ si $\epsilon \notin L$

Operaciones

Propiedades:

- $L^+ = L^*$ si $\epsilon \in L$
- $L^+ = L^* - \{\epsilon\}$ si $\epsilon \notin L$

Ejemplo

Si $L = \{0, 01\}$, entonces:

$L^* =$

$L^+ =$

Operaciones

Propiedades:

- $L^+ = L^*$ si $\epsilon \in L$
- $L^+ = L^* - \{\epsilon\}$ si $\epsilon \notin L$

Ejemplo

Si $L = \{0, 01\}$, entonces:

$L^* =$ Conjunto de palabras sobre $\{0, 1\}$ en las que un uno va siempre precedido de un cero.

$L^+ =$ Conjunto de palabras sobre $\{0, 1\}$ en las que un uno va siempre precedido de un cero y distintas de la palabra vacía.

Lenguaje inverso

El **lenguaje inverso** de L es el lenguaje dado por:

$$L^{-1} = \{u \mid u^{-1} \in L\}$$

Operaciones

La **cabecera** de L es el lenguaje dado por

$$\text{CAB}(L) = \{u \in A^* \mid \exists v \in A^* \text{ tal que } uv \in L\}$$

Ejemlo

Si $L = \{0^i 1^i : i \geq 0\}$,

Operaciones

La **cabecera** de L es el lenguaje dado por

$$\text{CAB}(L) = \{u \in A^* \mid \exists v \in A^* \text{ tal que } uv \in L\}$$

Ejemlo

Si $L = \{0^i 1^i : i \geq 0\}$, entonces $\text{CAB}(L) = \{0^i 1^j : i \geq j \geq 0\}$.

Operaciones

Si A_1 y A_2 son dos alfabetos, una aplicación

$$h : A_1^* \to A_2^*$$

se dice que es un **homomorfismo** si y solo si

$$h(uv) = h(u)h(v)$$

Consecuencias

- $h(\epsilon) = \epsilon$
- $h(a_1 \dots a_n) = h(a_1) \dots h(a_n)$

Homomorfismo

Si $A_1 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, A_2 = \{0, 1\}$

$$h(0) = 0000, \quad h(1) = 0001, \quad h(2) = 0010, \quad h(3) = 0011$$
$$h(4) = 0100, \quad h(5) = 0101, \quad h(6) = 0110, \quad h(7) = 0111$$
$$h(8) = 1000 \quad h(9) = 1001$$

$$h(034) = 000000110100, \qquad h(\epsilon) = \epsilon$$

Preguntas

¿Verdadero of falso?

- Si A es un afabeto, la aplicación que transforma cada palabra $u \in A^*$ en su inversa es un homomorfismo de A^* en A^*.

Preguntas

¿Verdadero of falso?

- Si A es un afabeto, la aplicación que transforma cada palabra $u \in A^*$ en su inversa es un homomorfismo de A^* en A^*.
  FALSO
- La transformación que a cada palabra sobre $\{0,1\}^*$ le añade 00 al principio y 11 al final es un homomorfismo.

Preguntas

¿Verdadero of falso?

- Si A es un afabeto, la aplicación que transforma cada palabra $u \in A^*$ en su inversa es un homomorfismo de A^* en A^*.
  FALSO
- La transformación que a cada palabra sobre $\{0,1\}^*$ le añade 00 al principio y 11 al final es un homomorfismo.
  FALSO

Contenido

Breve introducción histórica a la computación

Definiciones básicas y operaciones

Gramáticas

Jerarquía de Chomsky

Gramáticas generativas

Una **gramática generativa** es un cuadrupla (V, T, P, S) en la que

- V es un alfabeto, llamado de variables o símbolos no terminales. Sus elementos se suelen representar con letras mayúsculas.
- T es un alfabeto, llamado de símbolos terminales. Sus elementos se suelen representar con letras minúsculas.
- P es un conjunto finito de pares (α, β), llamados reglas de producción, donde $\alpha, \beta \in (V \cup T)^*$ y α contiene, al menos un símbolo de V.

  El par (α, β) se suele representar como $\alpha \to \beta$.
- S es un elemento de V, llamado símbolo de partida.

Gramática

$G = (V, T, P, S)$ dada por,

- $V = \{E\}$
- $T = \{+, *, (,), a, b, c\}$
- P está compuesto por las siguientes reglas de producción

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

- $S = E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

E

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (a + E) * E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (a + E) * E \Rightarrow$

$(a + b) * E$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (a + E) * E \Rightarrow$

$(a + b) * E \Rightarrow (a + b) * b$

Lenguaje generado: idea intuitiva

Una gramática sirve para determinar un lenguaje.

$$E \to E + E, \quad E \to E * E, \quad E \to (E),$$
$$E \to a, \quad E \to b, \quad E \to c$$

Las palabras son las de T^* que se obtienen a partir del símbolo inicial efectuando pasos de derivación. Cada paso consiste en elegir una parte de la palabra que coincide con la parte izquierda de una producción y sustituir esa parte por la derecha de la misma producción.

Ejemplo

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (a + E) * E \Rightarrow$

$(a + b) * E \Rightarrow (a + b) * b$ Palabra Generada

Paso de derivación

Gramática $G = (V, T, P, S)$ y dos palabras $\alpha, \beta \in (V \cup T)^*$

β es **derivable** a partir de α **en un paso** $(\alpha \Longrightarrow \beta)$

si y solo si

existe una producción $\gamma \to \varphi$ tal que

- α contiene a γ como subcadena.
- β se obtiene sustituyento γ por φ en α .

Secuencia de derivación

β es **derivable** de α $(\alpha \overset{*}{\Longrightarrow} \beta)$,

si y solo si existe una sucesión de palabras $\gamma_1, \ldots, \gamma_n$ $(n \geq 1)$

tales que

$$\alpha = \gamma_1 \Longrightarrow \gamma_2 \Longrightarrow \ldots \Longrightarrow \gamma_n = \beta$$

Lenguaje generado

Lenguaje generado por una gramática $G = (V, T, P, S)$ al conjunto de cadenas formadas por símbolos terminales y que son derivables a partir del símbolo de partida.

Es decir,

$$L(G) = \{u \in T^* \mid S \overset{*}{\Longrightarrow} u\}$$

Ejemplo

$G = (V, T, P, S)$, donde $V = \{S, A, B\}$, $T = \{a, b\}$, el símbolo de partida es S y las reglas son

$$S \to aB, \quad S \to bA, \quad A \to a, \quad A \to aS,$$
$$A \to bAA, \quad B \to b, \quad B \to bS, \quad B \to aBB$$

Ejemplo

$G = (V, T, P, S)$, donde $V = \{S, A, B\}$, $T = \{a, b\}$, el símbolo de partida es S y las reglas son

$$\begin{array}{llll} S \to aB, & S \to bA, & A \to a, & A \to aS, \\ A \to bAA, & B \to b, & B \to bS, & B \to aBB \end{array}$$

Esta gramática genera el lenguaje

$$L(G) = \{u \mid u \in \{a, b\}^+ \text{ y } N_a(u) = N_b(u)\}$$

donde $N_a(u)$ y $N_b(u)$ son el número de apariciones de símbolos a y b, en u, respectivamente.

Demostración

Esto es fácil de ver interpretando,

- A palabra con una a de más
- B palabra con una b de más
- S palabra con igual número de a que de b.

Hay que demostrar dos cosas:

- Todas las palabras generadas por la gramática tienen el mismo número de a que de b.
- Cualquier palabra con el mismo número de a que de b es generada.

Demostración

Para lo primero, podemos considerar $N_{a,A}(\alpha)$ (número de a + número de A) y $N_{b,B}(\alpha)$ (número de b + número de B) y tener en cuenta lo siguiente para una generación $S \overset{*}{\Longrightarrow} u$:

- Al principio de la generación tenemos: $N_{a,A}(S) = N_{b,B}(S) = 0$
- Al aplicar cualquier regla $\alpha_1 \Rightarrow \alpha_2$, si $N_{a,A}(\alpha_1) = N_{b,B}(\alpha_1)$, entonces $N_{a,A}(\alpha_2) = N_{b,B}(\alpha_2)$
- Luego al final $N_{a,A}(u) = N_{b,B}(u)$, y como u no contiene variables, $N_a(u) = N_b(u)$, como se quería demostrar.

Algoritmo de generación

Generación por la izquierda, un símbolo cada vez.

- Para generar una a
  - Si a último símbolo de la palabra, aplicar $A \to a$
  - Si no es el último símbolo
    - Si la primera variable es S aplicar $S \to aB$
    - Si la primera variable es B aplicar $B \to aBB$
    - Si la primera variable es A
      - Si hay más variables aplicar $A \to a$
      - Si no hay más, aplicar $A \to aS$
- Para generar una b
  - Si b último símbolo de la palabra, aplicar $B \to b$
  - Si no es el último símbolo
    - Si la primera variable es S aplicar $S \to bA$
    - Si la primera variable es A aplicar $A \to bAA$
    - Si la primera variable es B
      - Si hay más variables aplicar $B \to b$
      - Si no hay más, aplicar $B \to bS$

Condiciones de garantía

- Las palabras generadas tienen primero símbolos terminales y después variables.
- Se genera un símbolo de la palabra en cada paso de derivación
- Las variables que aparecen en la palabra pueden ser:
  - Una cadena de A (si hemos generado más b que a)
  - Una cadena de B (si hemos generado más a que b)
  - Una S si hemos generado las mismas a que b
- Antes de generar el último símbolo tendremos como variables:
  - Una A si tenemos que generar a
  - Una B si tenemos que generar b
- Entonces aplicamos la primera opción para generar los símbolos y la palabra queda generada.

Ejemplo

Sea $G = (\{S, X, Y\}, \{a, b, c\}, P, S)$ donde P tiene las reglas,

$$S \to abc \qquad S \to aXbc \qquad Xb \to bX \qquad Xc \to Ybcc$$
$$bY \to Yb \qquad aY \to aaX \qquad aY \to aa$$

Lenguaje generado

$$S \to abc \qquad S \to aXbc \qquad Xb \to bX \qquad Xc \to Ybcc$$
$$bY \to Yb \qquad aY \to aaX \qquad aY \to aa$$

Esta gramática genera el lenguaje: $\{a^n b^n c^n \mid n = 1, 2, \ldots\}$.

Inicialmente tenemos dos posibilidades:

$$S \Longrightarrow abc, \qquad S \Longrightarrow aXbc$$

Con la primera generamos la palabra abc.

Si seguimos la segunda opción, sólo se puede seguir cómo:

$$aXbc \Longrightarrow abXc \Longrightarrow abYbcc \Longrightarrow aYbbcc$$

Lenguaje generado: $\{a^n b^n c^n \mid n = 1, 2, \ldots\}$

$S \to abc$ $\quad S \to aXbc$ $\quad Xb \to bX$ $\quad Xc \to Ybcc$

$bY \to Yb$ $\quad aY \to aaX$ $\quad aY \to aa$

Ya tenemos abc.

Habíamos generado también: $aYbbcc$. En este momento podemos aplicar dos reglas:

- $aY \to aa$, en cuyo caso producimos $aabbcc = a^2b^2c^2 \in L(G)$
- $aY \to aaX$, en cuyo caso producimos $aaXbbcc$

A partir de $aaXbbcc$ repetimos el proceso: sólo se puede mover la X a la derecha hasta la frontera $b - c$. Entonces se añade una b y una c y se cambia X por Y.

Después, se mueve la Y a la izquierda hasta que encuentra la frontera $a - b$. Entonces, tiene dos opciones: añadir sólo a, obteniendo la siguiente palabra, o aX con lo que se vuelven a generar las otras palabras.

Contenido

Breve introducción histórica a la computación

Definiciones básicas y operaciones

Gramáticas

Jerarquía de Chomsky

Jerarquía de Chomsky

- **Tipo 0** Cualquier gramática. Sin restricciones. Lenguajes recursivamente enumerables.
- **Tipo 1** Si todas las producciones tienen la forma

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$$

donde $\alpha_1, \alpha_2, \beta \in (V \cup T)^*, A \in V$, y $\beta \neq \epsilon$, excepto posiblemente la regla $S \rightarrow \epsilon$, en cuyo caso S no aparece a la derecha de las reglas. Lenguajes dependientes del contexto.

Jerarquía de Chomsky

- **Tipo 2** Si cualquier producción tiene la forma

$$A \to \alpha$$

donde $A \in V, \alpha \in (V \cup T)^*$.
Lenguajes Independientes del Contexto

- **Tipo 3** Si toda regla tiene la forma

$$A \to uB \text{ ó } A \to u$$

donde $u \in T^*$ y $A, B \in V$.
Conjuntos Regulares

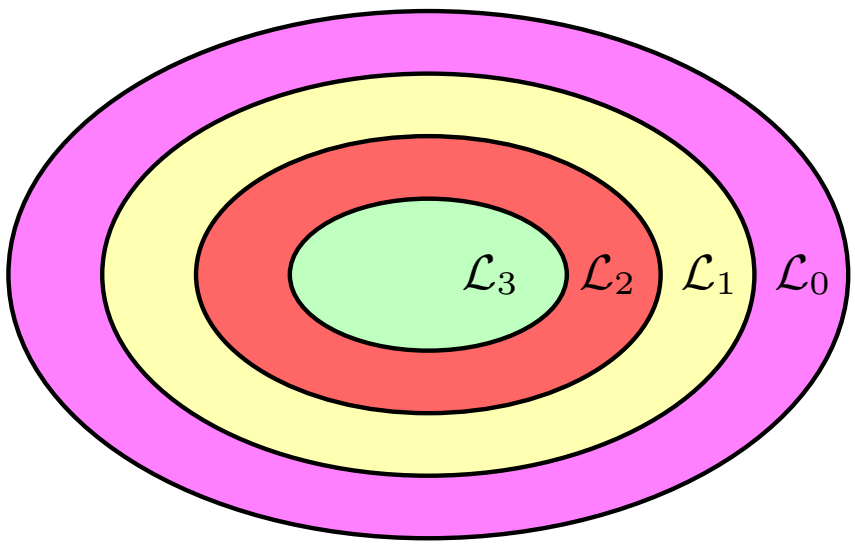
Clases de Lenguajes

Un lenguaje se dice que es de tipo i $(i = 0, 1, 2, 3)$ si y solo si es generado por una gramática de tipo i.

La clase o familia de lenguajes de tipo i se denota por $\mathcal{L}_i$.

Propiedad

$$\mathcal{L}_3 \subseteq \mathcal{L}_2 \subseteq \mathcal{L}_1 \subseteq \mathcal{L}_0$$



Ejercicios

Demostrar que la gramática

$$G = (\{S\}, \{a, b\}, \{S \to \epsilon, S \to \text{ aSb}\}, S)$$

genera el lenguaje $L = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$

Ejercicios

Demostrar que la gramática

$$G = (\{S\}, \{a, b\}, \{S \to \epsilon, S \to \text{ aSb}\}, S)$$

genera el lenguaje $L = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$

Inicialmente tenemos dos opciones:

$$S \Rightarrow \epsilon, \qquad S \Rightarrow aSb$$

Ejercicios

Demostrar que la gramática

$$G = (\{S\}, \{a, b\}, \{S \to \epsilon, S \to \text{ aSb}\}, S)$$

genera el lenguaje $L = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$

Inicialmente tenemos dos opciones:

$$S \Rightarrow \epsilon, \qquad S \Rightarrow aSb$$

Con eso generamos la palabra vacía, o continuamos generando. Otra vez hay dos opciones:

$S \Rightarrow aSb \Rightarrow ab, \qquad S \Rightarrow aSb \Rightarrow aaSbb$

Ejercicios

Demostrar que la gramática

$$G = (\{S\}, \{a, b\}, \{S \to \epsilon, S \to \text{ aSb}\}, S)$$

genera el lenguaje $L = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$

Si seguimos este procedimiento, nos encontramos que podemos ir generando todas las palabras de la forma $a^i b^i$, y siempre nos queda la palabra $a^i S b^i$ para seguir generando las palabras de mayor longitud.

Ejercicios

Demostrar que la gramática

$$G = (\{S\}, \{a, b\}, \{S \to \epsilon, S \to \text{ aSb}\}, S)$$

genera el lenguaje $L = \{a^i b^i \mid i = 0, 1, 2, \ldots\}$

Si seguimos este procedimiento, nos encontramos que podemos ir generando todas las palabras de la forma $a^i b^i$, y siempre nos queda la palabra $a^i S b^i$ para seguir generando las palabras de mayor longitud.

Por otra parte, estas son las únicas palabras que se pueden generar.

Ejercicios

Encontrar el lenguaje generado por la gramática $G = (\{A, B, C\}, \{a, b\}, P, S)$ donde P contiene las siguientes producciones

$$S \to aAB \qquad bB \to a \qquad Ab \to SBb$$
$$Aa \to SaB \qquad B \to SA \qquad B \to ab$$

Ejercicios

Encontrar el lenguaje generado por la gramática $G = (\{A, B, C\}, \{a, b\}, P, S)$ donde P contiene las siguientes producciones

$$S \to aAB \qquad bB \to a \qquad Ab \to SBb$$
$$Aa \to SaB \qquad B \to SA \qquad B \to ab$$

El resultado es el Lenguaje vacío: nunca se puede llegar a generar una palabra con símbolos terminales. Siempre que se sustituye S aparece A, y siempre que se sustituye A aparece S.

Ejercicios

Encontrar una gramática libre del contexto para generar cada uno de los siguientes lenguajes

1. $L = \{a^i b^j \mid i, j \in \mathbb{N}, i \leq j\}$
2. $L = \{a^i b^j a^j b^i \mid i, j \in \mathbb{N}\}$
3. $L = \{a^i b^i a^j b^j \mid i, j \in \mathbb{N}\}$
4. $L = \{a^i b^i \mid i \in \mathbb{N}\} \cup \{b^i a^i \mid i \in \mathbb{N}\}$
5. $L = \{uu^{-1} \mid u \in \{a, b\}^*\}$
6. $L = \{a^i b^j c^{i+j} \mid i, j \in \mathbb{N}\}$

donde $\mathbb{N}$ es el conjunto de los números naturales incluyendo el 0.

$L = \{a^i b^j \mid i, j \in \mathbb{N}, i \leq j\}$

$L = \{a^i b^j \mid i, j \in \mathbb{N}, i \leq j\}$

$$S \to aSb$$

$$S \to \epsilon$$

$L = \{a^i b^j \mid i, j \in \mathbb{N}, i \leq j\}$

$$S \to aSb$$

$$S \to \epsilon$$

$$S \to Sb$$

$L = \{a^i b^j a^j b^i \mid i, j \in \mathbb{N}\}$

Este lenguaje es generado por la siguiente gramática:

$L = \{a^i b^j a^j b^i \mid i, j \in \mathbb{N}\}$

Este lenguaje es generado por la siguiente gramática:

$$S \to aSb$$

$$S \to B, \quad B \to bBa, \quad B \to \epsilon$$

$L = \{a^i b^i a^j b^j \mid i, j \in \mathbb{N}\}$

Podemos generar $\{a^i b^i \mid i \in \mathbb{N}\}$ con:

$L = \{a^i b^i a^j b^j \mid i, j \in \mathbb{N}\}$

Podemos generar $\{a^i b^i \mid i \in \mathbb{N}\}$ con:

$$S_1 \to aS_1b, \quad S_1 \to \epsilon$$

$L = \{a^i b^i a^j b^j \mid i, j \in \mathbb{N}\}$

Podemos generar $\{a^i b^i \mid i \in \mathbb{N}\}$ con:

$$S_1 \to aS_1b, \quad S_1 \to \epsilon$$

El lenguaje L se puede generar añadiendo:

$$S \to S_1 S_1$$

siendo S el símbolo inicial.

$$L = \{a^i b^i \mid i \in \mathbb{N}\} \cup \{b^i a^i \mid i \in \mathbb{N}\}$$

$L = \{a^i b^i \mid i \in \mathbb{N}\} \cup \{b^i a^i \mid i \in \mathbb{N}\}$

Podemos generar $\{a^i b^i \mid i \in \mathbb{N}\}$ con:

$$S_1 \to aS_1b, \quad S_1 \to \epsilon$$

y $\{b^i a^i \mid i \in \mathbb{N}\}$ con

$$S_2 \to bS_2a, \quad S_2 \to \epsilon$$

$L = \{a^i b^i \mid i \in \mathbb{N}\} \cup \{b^i a^i \mid i \in \mathbb{N}\}$

Podemos generar $\{a^i b^i \mid i \in \mathbb{N}\}$ con:

$$S_1 \to aS_1b, \quad S_1 \to \epsilon$$

y $\{b^i a^i \mid i \in \mathbb{N}\}$ con

$$S_2 \to bS_2a, \quad S_2 \to \epsilon$$

El lenguaje L se puede generar añadiendo:

$$S \to S_1, \quad S \to S_2$$

siendo S el símbolo inicial.

$L = \{uu^{-1} \mid u \in \{a, b\}^*\}$

Este lenguaje se genera con la gramática:

$L = \{uu^{-1} \mid u \in \{a, b\}^*\}$

Este lenguaje se genera con la gramática:

$$S \to aSa, \quad S \to bSb, \quad S \to \epsilon$$

$L = \{a^i b^j c^{i+j} \mid i, j \in \mathbb{N}\}$

Este lenguaje se genera con la gramática:

$L = \{a^i b^j c^{i+j} \mid i, j \in \mathbb{N}\}$

Este lenguaje se genera con la gramática:

$$S \to aSc, \quad S \to B,$$

$$B \to bBc, \quad B \to \epsilon$$

Ejercicios

Determinar si la gramática $G = (\{S, A, B\}, \{a, b, c, d\}, P, S)$ donde P es el conjunto de reglas de producción:

$$\begin{array}{lll} S \to AB & A \to Ab & A \to a \\ B \to cB & B \to d & \end{array}$$

genera un lenguaje de tipo 3.

Ejercicios

Determinar si la gramática $G = (\{S, A, B\}, \{a, b, c, d\}, P, S)$ donde P es el conjunto de reglas de producción:

$$\begin{array}{lll} S \to AB & A \to Ab & A \to a \\ B \to cB & B \to d & \end{array}$$

genera un lenguaje de tipo 3.

Esta gramática genera el lenguaje: $\{ab^i c^j d \ : \ i, j \in \mathbb{N}\}$

Ejercicios

Determinar si la gramática $G = (\{S, A, B\}, \{a, b, c, d\}, P, S)$ donde P es el conjunto de reglas de producción:

$$\begin{array}{lll} S \to AB & A \to Ab & A \to a \\ B \to cB & B \to d & \end{array}$$

genera un lenguaje de tipo 3.

Esta gramática genera el lenguaje: $\{ab^i c^j d \,:\, i, j \in \mathbb{N}\}$

Y este lenguaje se puede generar mediante la gramática:

$S \to aB, \quad B \to bB, \quad B \to C, \quad C \to cC, \quad C \to d$

Como esta gramática es de tipo 3, el lenguaje lo es.